



NAME : HAMDAN ALI

PROJECT :- TO-DO List App

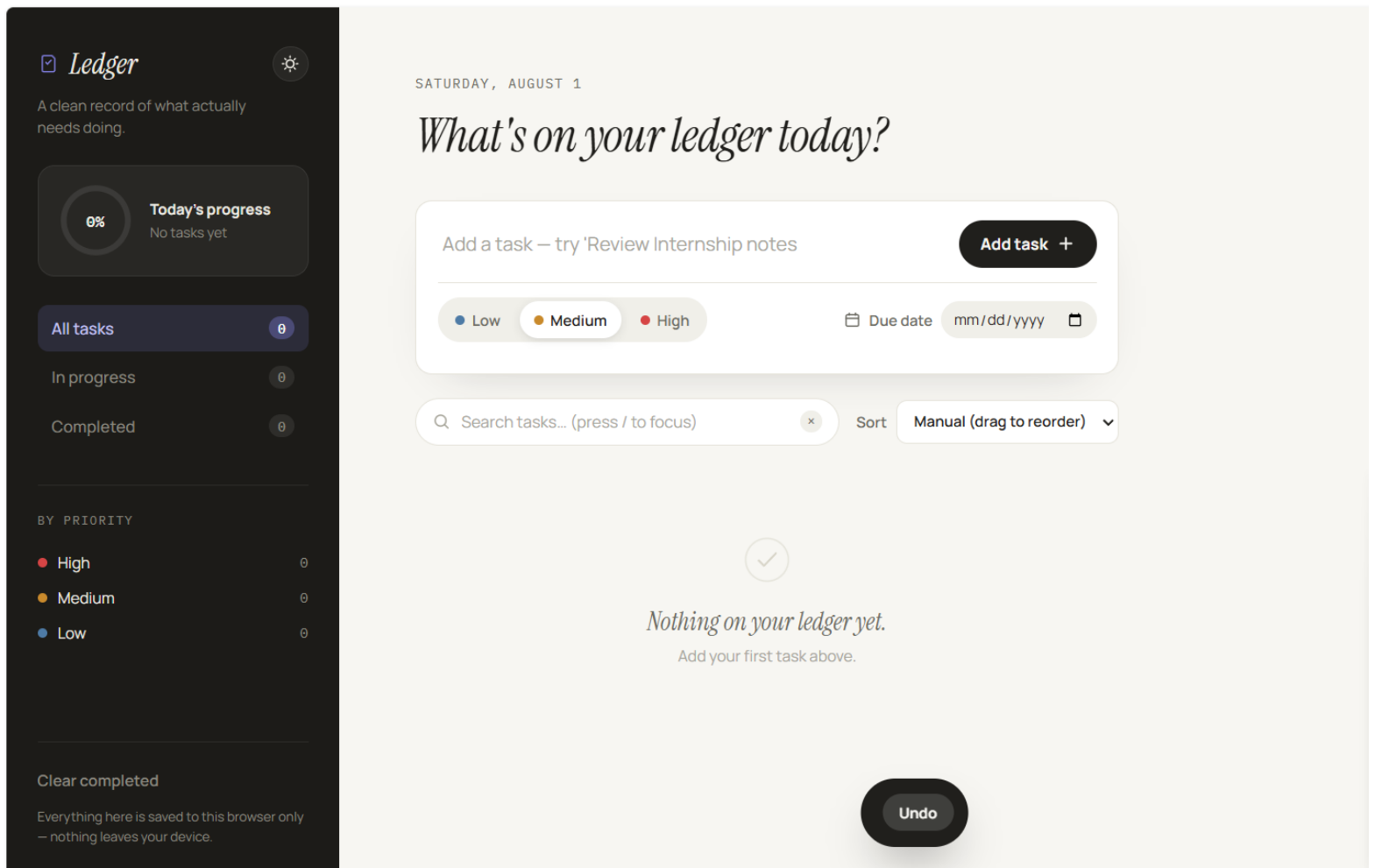
WEEK 4 : Day 20

DAY 20

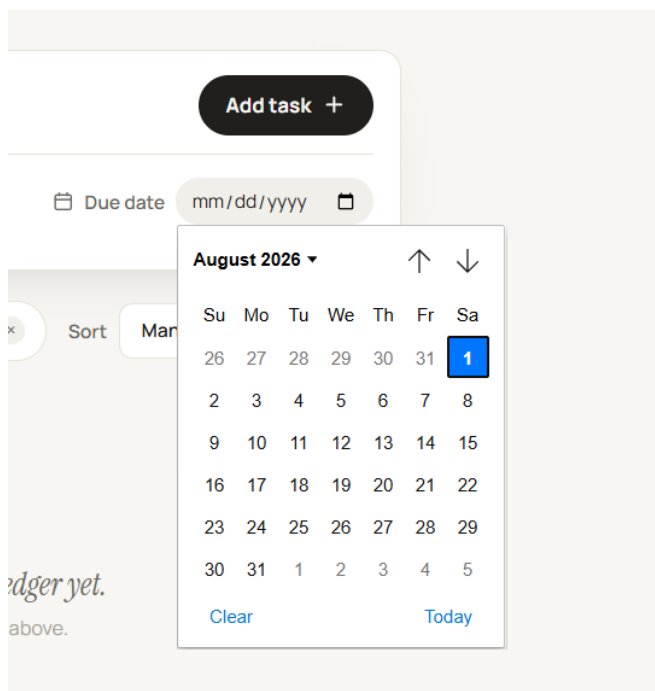
Build a To-Do List App with add, delete, and save functionality.

Project : Screenshots Of Output

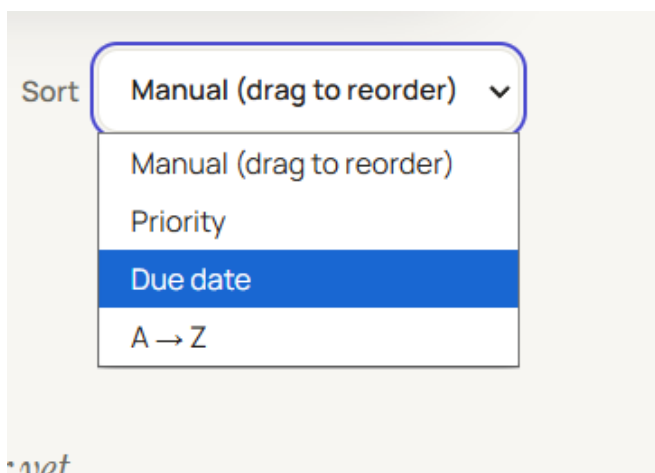
Full Project View



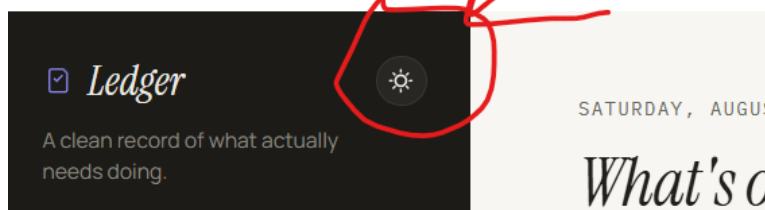
Date Setting

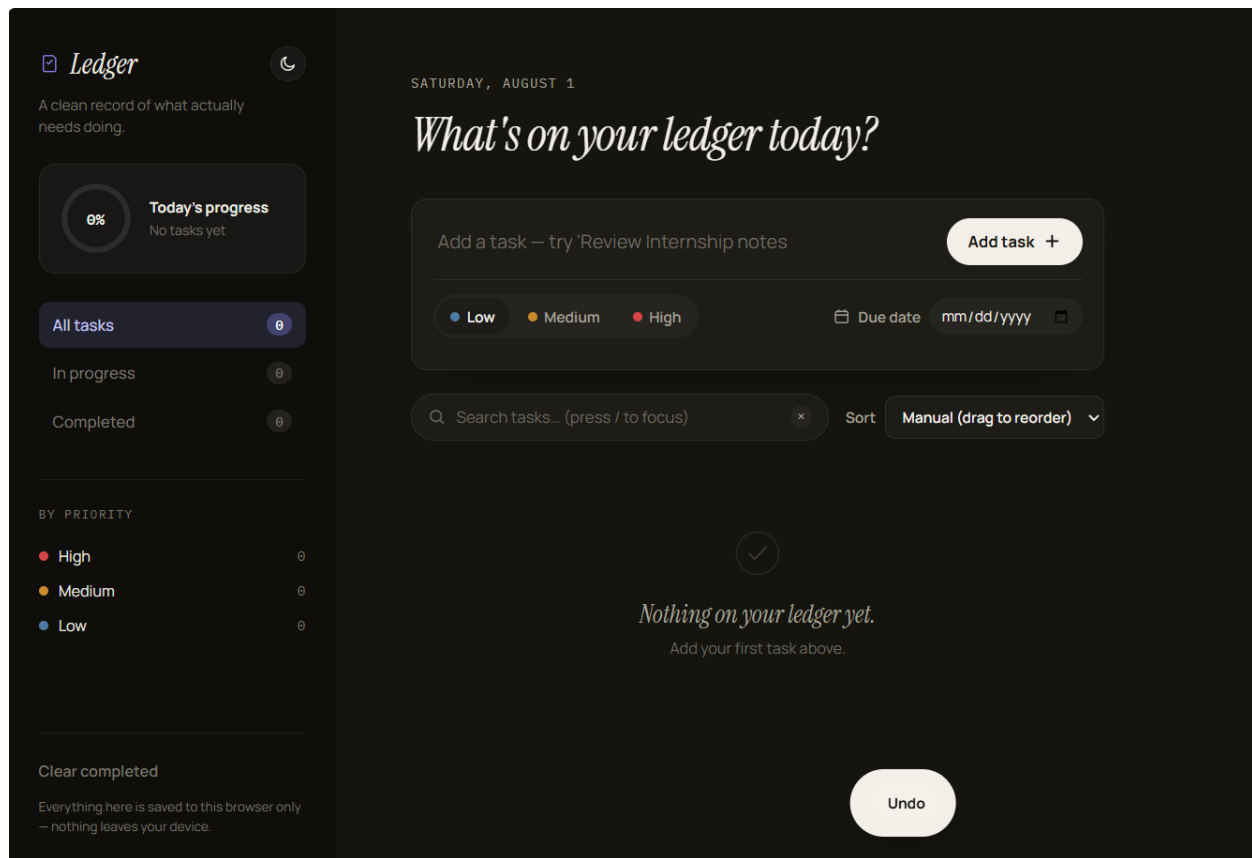


Manual

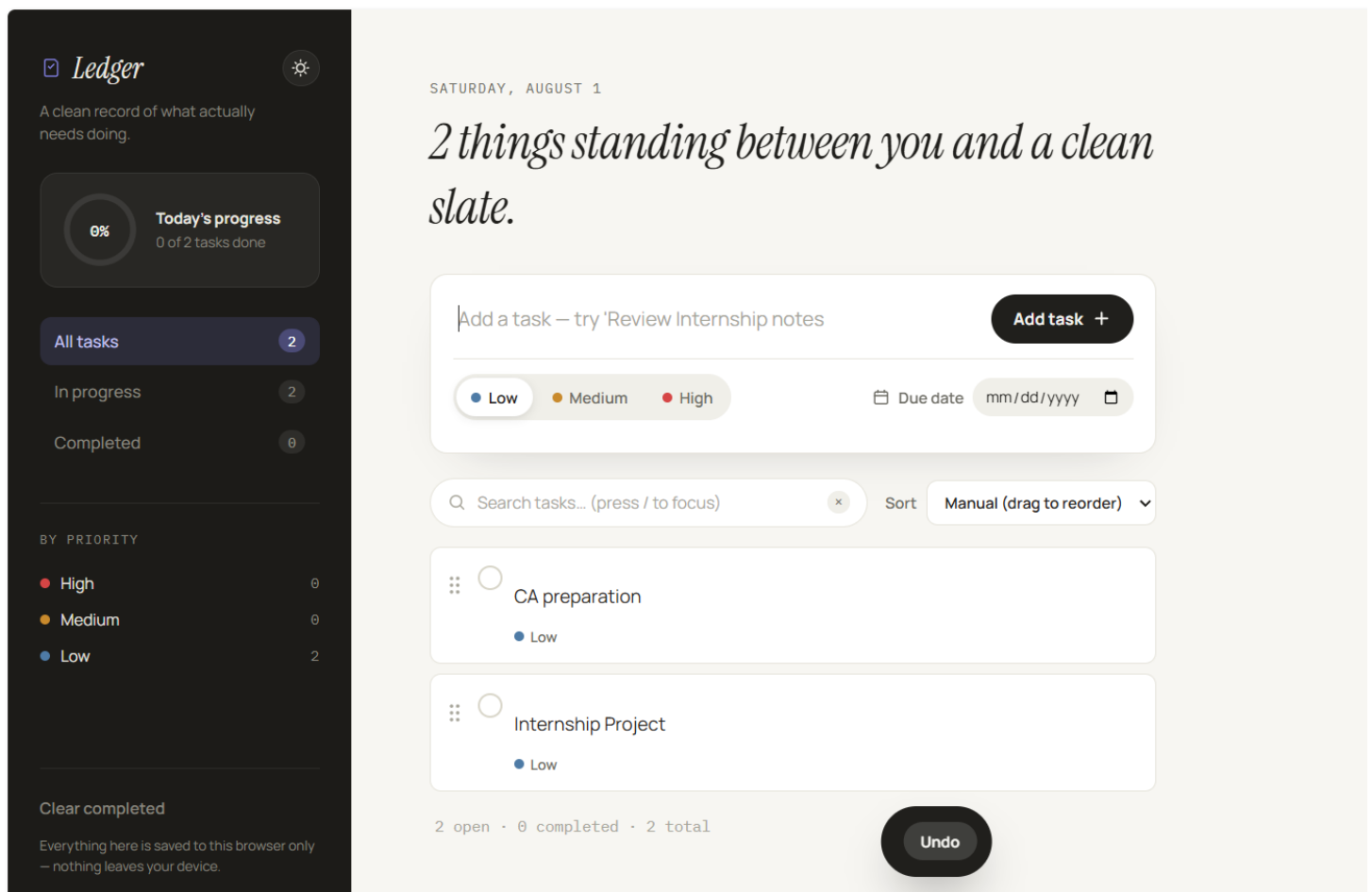


We can change the theme to dark theme

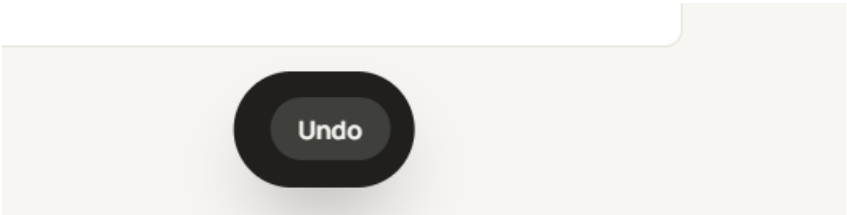




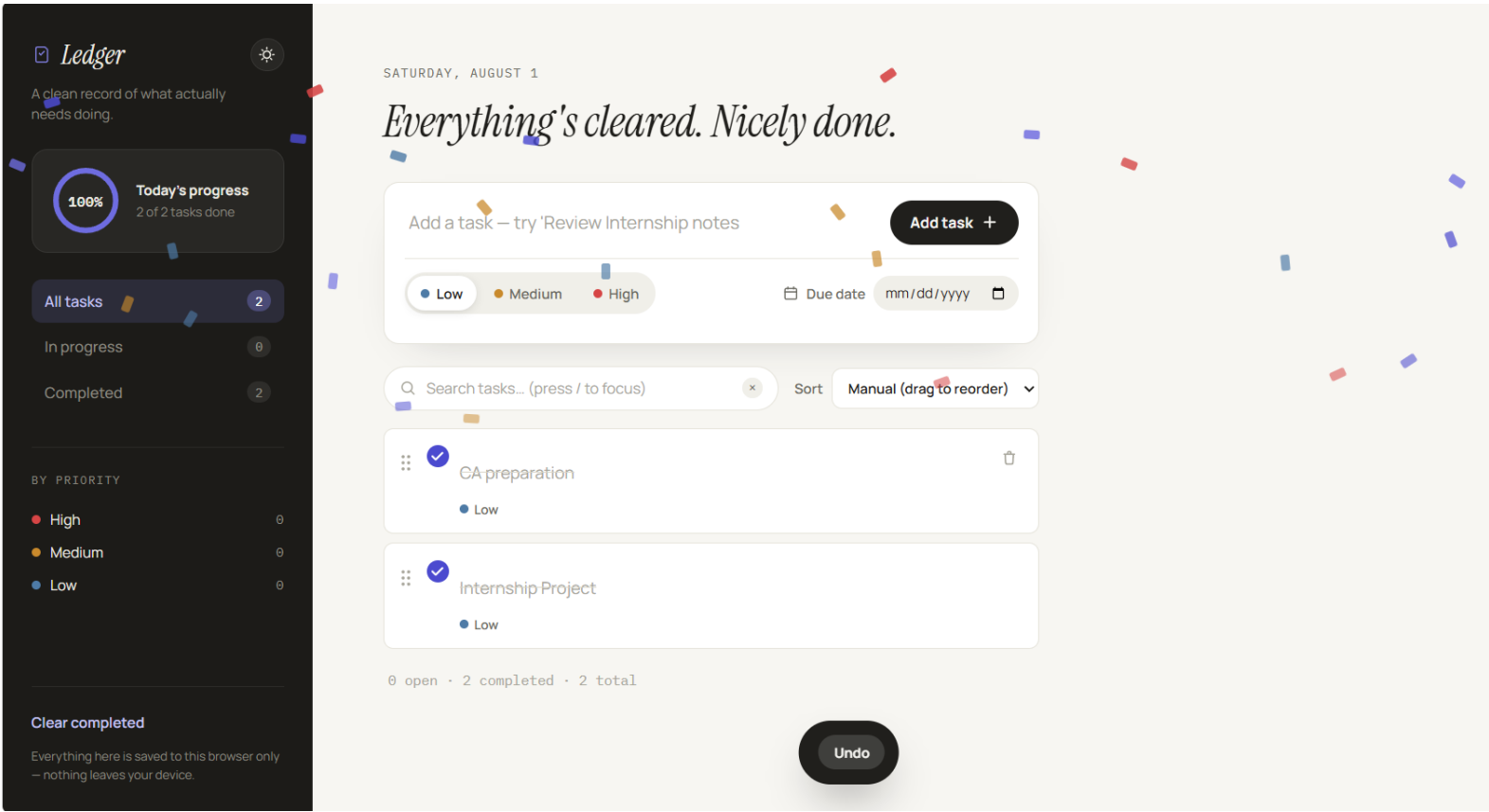
After Adding Tasks



The Undo Toast Button is for if delete a Task then if we need it back the Undo Toast Button help us



Celebrating When the Tasks are Completed



In The below image I highlighted the reordering and the level complexity of the task that is low ,medium or high

Add a task – try 'Review Internship notes'

Add task +

Low

Medium

High

Due date mm/dd/yyyy

Search tasks... (press / to focus)

Sort Manual (drag to reorder)

Internship Project

Low

CA preparation

Low

0 open · 2 completed · 2 total



EXPLANATION OF THE PROJECT

DAY 20 — TO-DO LIST APP

Building Ledger

Hamdan Ali · Week 4 · Full Stack Web Development Internship

What I Was Going For

The brief was add, delete, and save — which is genuinely the smallest possible to-do list, and I could have built exactly that in about thirty lines. Before writing any of it, I spent a while actually using the to-do apps people rely on day to day — Todoist, TickTick, Things 3 — and reading through comparisons of them, because it felt wrong to build a task manager without knowing what the good ones actually get right and where people complain about the bad ones.

A few things kept coming up. Fast capture matters more than almost any other feature — if adding a task takes more than a couple of seconds, people stop using the app. Things 3 gets singled out constantly for how calm it feels, not for having the most features. And nobody mentions save buttons, because in a good app saving isn't a separate action at all — it just happens, silently, the moment you do anything. I tried to build Ledger around those three ideas specifically, rather than around a checklist of features.

Why It's Not Just a List

Ledger ended up with a sidebar and a main panel rather than a single column, because that's the layout every serious task manager converges on for a reason — it lets you see where you stand (how many tasks, how many left, how many by priority) without that information competing for space with the tasks themselves. The sidebar stays dark and quiet; the task list stays light and is where your attention actually goes.

Every task carries a priority and an optional due date, because a flat list of text with no way to tell what matters most is exactly the kind of to-do app people abandon after a week. Priority shows up as a small colored dot rather than a loud badge, and a due date only appears as a chip if you actually

set one — nothing forces you to fill in fields you don't care about, which was one of the specific complaints I kept seeing about heavier apps.

Fast Capture, Actually Fast

The add form is one text input, one button, and Enter submits it — priority and due date default sensibly (medium, no date) so you're never blocked from adding something just because you haven't decided how important it is yet:

```
composer.addEventListener('submit', (event) => {
  event.preventDefault();
  const text = taskInput.value.trim();
  if (!text) {
    composerError.textContent = 'Write something before adding it to the list.';
    composerError.classList.add('is-visible');
    return;
  }
  const task = {
    id: makeId(), text, priority: selectedPriority,
    dueDate: dueInput.value || null, completed: false,
    createdAt: new Date().toISOString(),
  };
  tasks.unshift(task);
  saveTasks();
  render();
});
```

The one validation rule here is the same empty-input check the earlier form-handling work covered — you can't add a blank task — but that's deliberately the only friction in the whole flow. Typing a task and pressing Enter should feel like writing it on paper, not like filling out a form.

Save Isn't a Button — It's Everything

There's no save button anywhere in Ledger, because every single change writes to localStorage the moment it happens — adding a task, checking one off, editing its text, deleting it, clearing completed tasks. All of it funnels through the same two functions:

```
function loadTasks() {
  const raw = localStorage.getItem(STORAGE_KEY);
  if (!raw) return [];
  try {
    const parsed = JSON.parse(raw);
    return Array.isArray(parsed) ? parsed : [];
  } catch (err) {
    localStorage.removeItem(STORAGE_KEY);
    return [];
  }
}

function saveTasks() {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(tasks));
}
```

This is the same single-key, try/catch-guarded pattern I'd used for a smaller storage exercise earlier in the week, and it held up well here at a larger scale — an array of task objects instead of one profile object, but the exact same reasoning about what happens when the saved data can't be trusted.

Deleting With an Actual Undo

This is the piece I'm most pleased with. Every to-do app I looked at during the research phase gives you a way to undo a delete, because permanently losing a task you tapped by accident is a genuinely bad experience — and almost no tutorial version of a to-do list bothers to include it. Deleting a task here removes it from the list and from storage immediately, but keeps a copy of it and its original position around for a few seconds while a toast offers to bring it back:

```
function deleteTask(id) {
  const index = tasks.findIndex((t) => t.id === id);
  const [removed] = tasks.splice(index, 1);
  saveTasks();
  render();
  showUndoToast(removed, index);
}
```



```
}

undoBtn.addEventListener('click', () => {
  if (!pendingDeletion) return;
  clearTimeout(pendingDeletion.timerId);
  const { task, index } = pendingDeletion;
  tasks.splice(Math.min(index, tasks.length), 0, task);
  saveTasks();
  render();
});
```

The toast disappears on its own after five seconds via `setTimeout`, and clicking Undo cancels that timer and splices the task back into the exact index it came from, rather than just appending it to the end of the list. Getting that index right — capturing it before the splice that removes the task, not after — was the one bug I actually hit while building this, and it's the kind of small detail that's invisible when it works and obviously broken when it doesn't.

Editing in Place

Double-clicking a task's text turns it into an editable field right where it sits, rather than opening a separate edit form. It commits on blur or Enter, and Escape reverts to the original text without saving anything — a pattern lifted directly from how Things 3 and Todoist both handle quick renames, and one that turned out to be more fiddly than it looks, mostly around making sure the cursor actually lands inside the text when editing starts:

```
function beginEdit(id, textEl) {
  textEl.contentEditable = 'true';
  textEl.focus();
  const range = document.createRange();
  range.selectNodeContents(textEl);
  window.getSelection().addRange(range);
  textEl.addEventListener('blur', () => {
    const newText = textEl.textContent.trim();
    textEl.contentEditable = 'false';
    if (newText) { task.text = newText; saveTasks(); }
  });
}
```

```
render();  
}, { once: true });  
}
```

Completed Tasks Sink, They Don't Vanish

Checking a task off doesn't delete it or hide it — it moves to the bottom of the list, gets a strikethrough, and fades in color, while staying fully visible and reachable. That was a deliberate choice after noticing how satisfying it is in other apps to watch a finished task visibly leave the "active" area without it disappearing outright, as if it just got out of your way rather than being erased. The sort itself is simple — open tasks first in the order they were added, completed tasks after:

```
function visibleTasks() {  
  const filtered = tasks.filter(/* current filter */);  
  const open = filtered.filter((t) => !t.completed);  
  const done = filtered.filter((t) => t.completed);  
  return [...open, ...done];  
}
```

Finding Things: Live Search

Once a list has more than a handful of tasks on it, scrolling to find one stops being reasonable — every app in the research phase had a search box for exactly this reason, so Ledger has one too, sitting just above the list. It filters on every keystroke rather than waiting for Enter, and the matching part of each task's text gets wrapped in a `<mark>` so it's obvious at a glance why a given task matched:

```
searchInput.addEventListener('input', () => {  
  searchQuery = searchInput.value.trim();  
  clearSearchBtn.hidden = searchQuery.length === 0;  
  render();  
});
```

```
document.addEventListener('keydown', (event) => {  
  const isTyping = /* focused element is a text field */;  
  if (event.key === '/' && !isTyping) {  
    event.preventDefault();  
    searchInput.focus();  
  }  
});
```

The "/" shortcut to jump into search is borrowed directly from how Todoist and most fast, keyboard-friendly tools behave — it's a small thing, but once you're used to it, clicking into a search box with a mouse feels slow by comparison.

Reordering Things: Sort Modes and Drag-and-Drop

The sort dropdown offers four views of the same data — Manual, Priority, Due date, and Alphabetical — and only Manual mode allows dragging, since reordering by hand only makes sense when nothing else is already dictating the order. Switching into Priority or Due-date mode hides the drag handles entirely rather than leaving them there and non-functional, which matters more than it sounds like — a drag handle that doesn't do anything is a worse experience than no handle at all.

The drag-and-drop itself uses the browser's native HTML5 drag events rather than a library, and reorders the underlying array live as you drag over each row, so by the time you actually release the mouse, the array is already correct and the drop handler just has to save it:

```
li.addEventListener('drop', (event) => {  
  if (sortMode !== 'manual' || !dragState || dragState.id === id) return;  
  event.preventDefault();  
  
  const fromIndex = tasks.findIndex((t) => t.id === dragState.id);  
  const toIndex = tasks.findIndex((t) => t.id === id);  
  const [moved] = tasks.splice(fromIndex, 1);  
  tasks.splice(toIndex, 0, moved);  
  render();
```

```
});
```

The save itself happens in `dragend` rather than in every single drop, since a drag gesture can pass over several rows before it lands — writing to `localStorage` once at the end of the gesture instead of on every intermediate reorder keeps it from doing unnecessary work while you're still deciding where something belongs.

A Ring That Actually Moves: Tracking Progress

The sidebar's progress ring is a plain SVG circle with its `stroke-dasharray` set to the circle's circumference and its `stroke-dashoffset` animated based on how much of the ledger is complete — no chart library, just a bit of geometry and a CSS transition on the offset so it eases into place instead of jumping:

```
const fraction = total === 0 ? 0 : done / total;
const offset = RING_CIRCUMFERENCE * (1 - fraction);
progressRing.style.strokeDashoffset = String(offset);
progressPercent.textContent = `${Math.round(fraction * 100)}%`;
```

It's a small piece of UI, but it turns "how am I doing today" into something you can read in half a second, without doing the arithmetic on the task counts yourself.

A Mode for the Evening: Dark Theme

Ledger remembers whether you left it in light or dark mode the same way it remembers your tasks — one more small key in `localStorage`, read synchronously in a blocking script before the stylesheet loads, so switching to dark mode and coming back later doesn't flash the light theme first:

```
(function () {
  var theme = localStorage.getItem('ledger:theme') || 'light';
  document.documentElement.setAttribute('data-theme', theme);
})();
```

Every color in the stylesheet is written as a CSS custom property scoped under [data-theme="light"] or [data-theme="dark"], so the toggle itself is just flipping one attribute — nothing in script.js needs to know what a color actually is.

The One Purely Delightful Feature: Celebrating a Clean Slate

This feature does nothing functional at all, and I built it anyway. The moment the last open task on the ledger gets checked off — not on page load, not from clearing the list, specifically from finishing the last thing — a short burst of confetti falls across the screen and the headline changes to acknowledge it:

```
const remainingBefore = tasks.filter((t) => !t.completed).length;
task.completed = !task.completed;
const remainingAfter = tasks.filter((t) => !t.completed).length;

if (remainingBefore > 0 && remainingAfter === 0) {
  celebrate();
}
```

Comparing the count before and after the toggle, rather than just checking if it's now zero, is what keeps this from firing on every page load for someone who already has an empty list — it only fires on the actual transition from "something left" to "nothing left," which is the one moment it's actually earned.

A Few Notes on What I Took From This

- Looking at real products before building the "same" feature changed what I actually built — an undo toast and a due-date chip aren't in the original task description anywhere, they came from noticing what the apps people actually use all have in common.
- "Save" doesn't have to be a button. Once every state change writes through one function, the whole app behaves like it's always saved, which is a better feeling than a save button ever gives you.
- Undo is a genuinely different feature from delete, not a smaller version of it — it needs the removed data and its original position kept alive on purpose for a short window, which



meant thinking about state that exists temporarily and on a timer, not just state that's permanently on or off.

- Building the task rows as real DOM elements instead of injected HTML strings meant every checkbox, delete button, and editable text field could have its event listener attached directly and safely, with no risk of a task's own text accidentally being interpreted as markup.

References Used

The W3Schools to-do list walkthrough covers the core DOM pattern this app is built on — creating list items, attaching click handlers, and removing nodes on delete. The video was useful for seeing that same DOM logic run end to end in a working example before I extended it with persistence, priorities, and the undo flow.

Article: [w3schools.com/howto/howto_js_todolist.asp](https://www.w3schools.com/howto/howto_js_todolist.asp)

Video: youtube.com/watch?v=ExDsHljnCyE

Files Submitted

- index.html — the sidebar (filters, priority breakdown, progress ring, theme toggle), the composer, the search-and-sort toolbar, the list, the undo toast, and the confetti layer.
- style.css — the full light/dark design system, the progress ring geometry, task and drag states, and the toast and confetti animations.
- script.js — persistence, rendering, priority and due-date handling, inline editing, search, sorting, native drag-and-drop reordering, the undo-on-delete logic, the theme toggle, and the completion celebration.

THANK YOU!